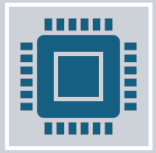
AI, GenAI & Agentic AI for Automotive Leaders and Developers

Surendra Panpaliya

🔧 Prerequisites

Basic knowledge of Python (functions, control flow)

Familiarity with automotive workflows,



systems, and product lifecycles

Lab Setup Requirements

- Laptop with Python 3.10+, Jupyter Notebook, VS Code
- Libraries: numpy, pandas, scikit-learn, matplotlib, openai, langchain, chromadb
- Access to Azure OpenAI or Google Gemini APIs
- Internet connectivity, Excel, and sample automotive datasets

🎓 Trainer Deliverables

- Pre-reading PDFs on AI, GenAI, Prompting
- Setup guide for labs (for technical team)
- Sample notebooks and visual examples
- Hackathon guidance
- Final review and open Q&A

Program Outcomes for Management Teams

Clearly understand AI, GenAI, and Agentic AI in business context

Evaluate where to invest in AI inside your automotive enterprise

Identify use cases with measurable ROI

Build leadership confidence in leveraging AI responsibly

Align technical and business teams on AI vision

Day-Wise Outline

Day 1: AI/ML/DL – Foundation for Automotive Innovation

Day 2: AI Leadership, Risk Management & Storytelling with Data

Day 3: Generative AI and Document Automation

Day 4: Strategic Thinking – Competing in the AI Age

Day 5: Introduction to Agentic AI and Innovation Hackathon

Day 5

Introduction to Agentic AI and

Innovation Hackathon

Objective: Introduce leaders to AI agents and

how they automate decision-making and tasks.

Day 5

What is Agentic AI (Simple Explanation)

Agents, Planners, Executors

Business Role View

Popular Tools: AutoGen, LangGraph

(only conceptual overview)

Day 5

Use Case: AI Assistant for Vehicle Configurations

Demo: Ask an AI agent about car specs

and receive tailored suggestions

Mini Hackathon: Team showcase of AI ideas

(no coding required for leaders)

What is Agentic AI?

Agentic AI = Smart AI agents that

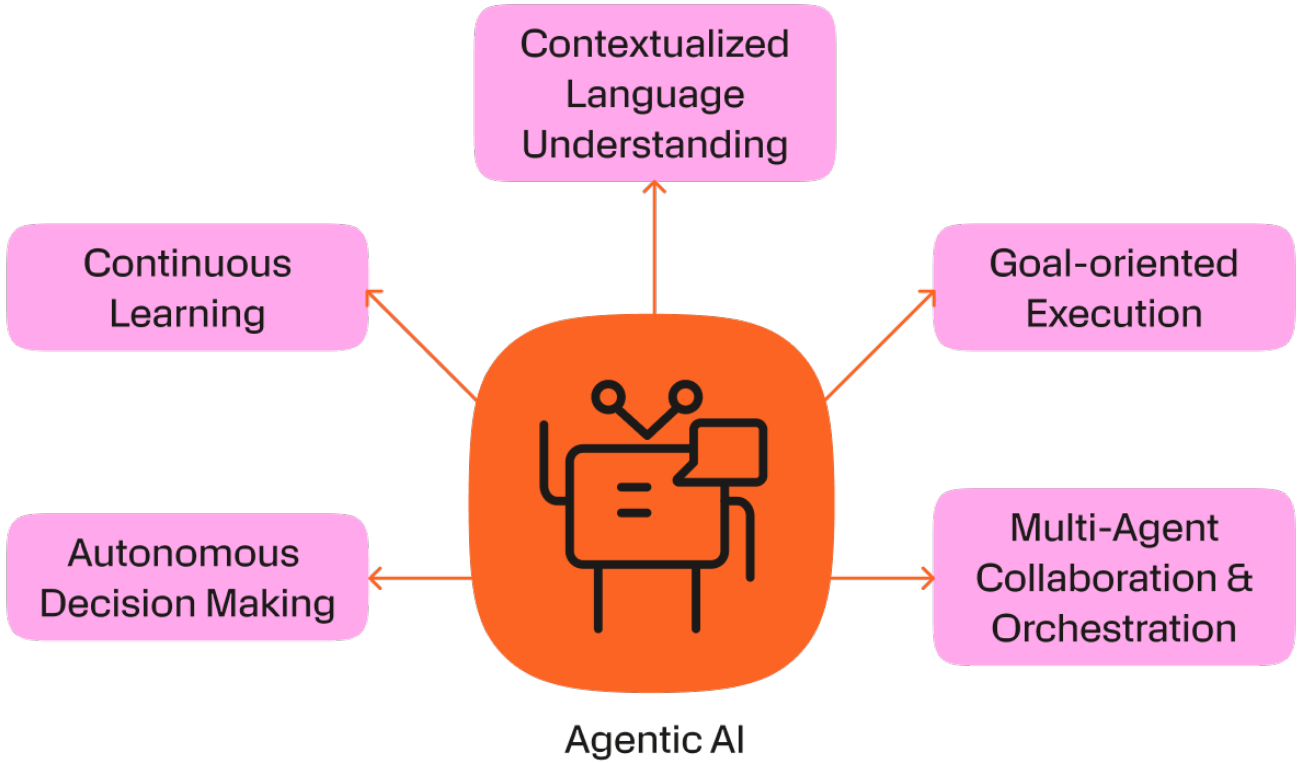
Understand goals

Plan steps

Execute tasks

Work together (like a team)

What is Agentic AI?



What is Agentic AI?

New way of using AI

to **plan, execute, and manage tasks**

independently

like an intelligent assistant

that thinks and acts.

What is Agentic AI?

ACT LIKE A RESPONSIBLE ASSISTANT,

DOING TASKS **INDEPENDENTLY**

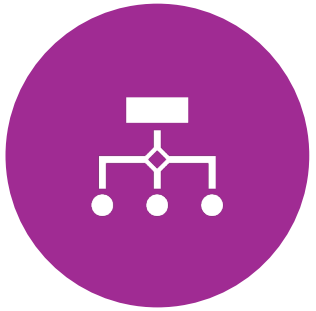
BASED ON GOALS YOU GIVE IT.

What is Agentic AI?

PLANS, DECIDES,

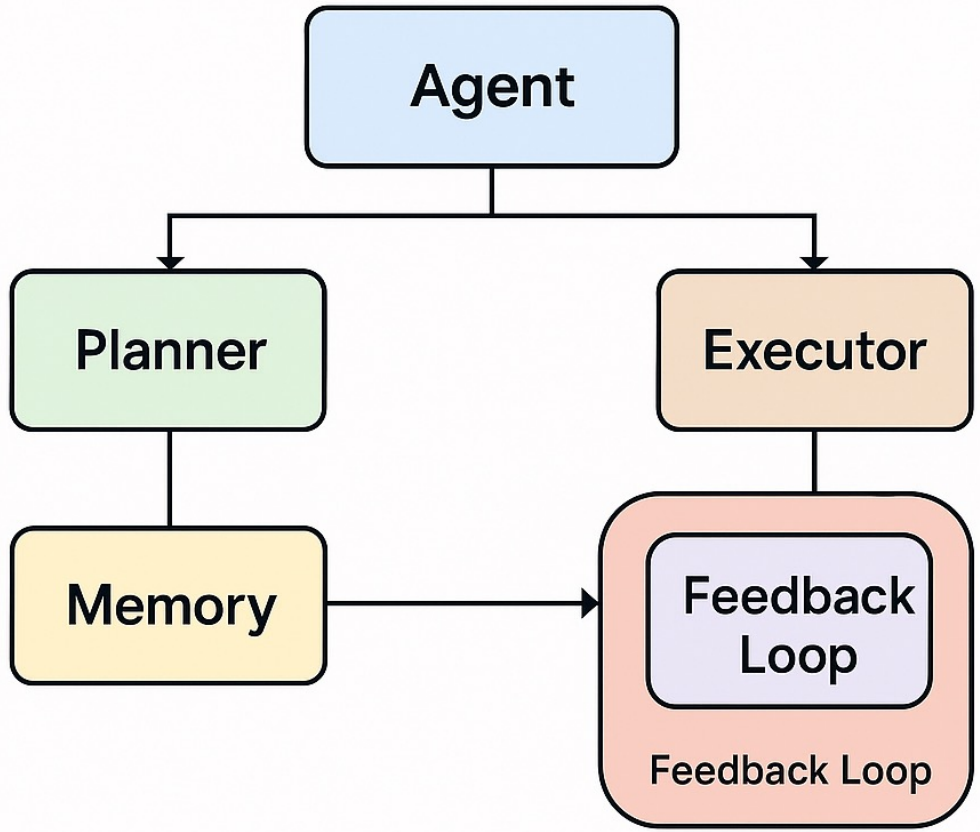
AND EXECUTES ACTIONS

TO SOLVE COMPLEX PROBLEMS OR



AUTOMATE MULTI-STEP TASKS.

AGENTIC ARCHITECTURE



👨‍💼 Agent

LIKE A **SMART EMPLOYEE**.

GIVE A GOAL

FIGURES OUT **WHAT TO DO**

HOW TO DO IT

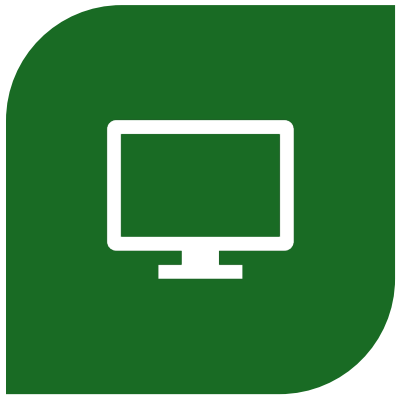
BY BREAKING IT INTO SUBTASKS.

Role


Understand the goal,
Coordinate the process
Manage tools or
Other sub-agents.

Example

BUILD AND DEPLOY



PYTHON WEB APP

ON AWS.

Planner

STRATEGIST INSIDE THE AGENT.

BREAKS DOWN THE GOAL INTO STEPS

CREATES A **WORKFLOW OR PLAN**

ACHIEVE THE GOAL.

Role

Task breakdown

Sequencing

Error handling

Example

CHECK FOR MISSING DEPENDENCIES.

RUN UNIT TESTS.

BUILD DOCKER IMAGE.

PUSH TO CONTAINER REGISTRY.

DEPLOY TO AWS ECS.

Executor

Runs the actual commands

Code, or API calls

Needed to perform

Each step of the plan

Role

Executes tasks

Like running scripts,

Querying APIs,

Committing code

Example

Use AWS CLI

To deploy

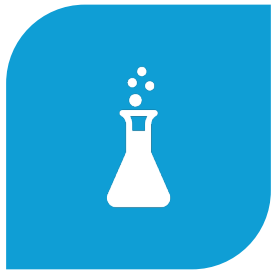
The Docker Container

To ECS.

Example

YOUR TEAM IS

BUILDING AND DEPLOYING



FLASK API ON **AWS ECS**

USING GITHUB AND DOCKER.

Use Case

AUTOMATING DEV

TEST

DEPLOY PIPELINE

WITH AGENTIC AI

👨‍💻 *Agent Task*

DEPLOY

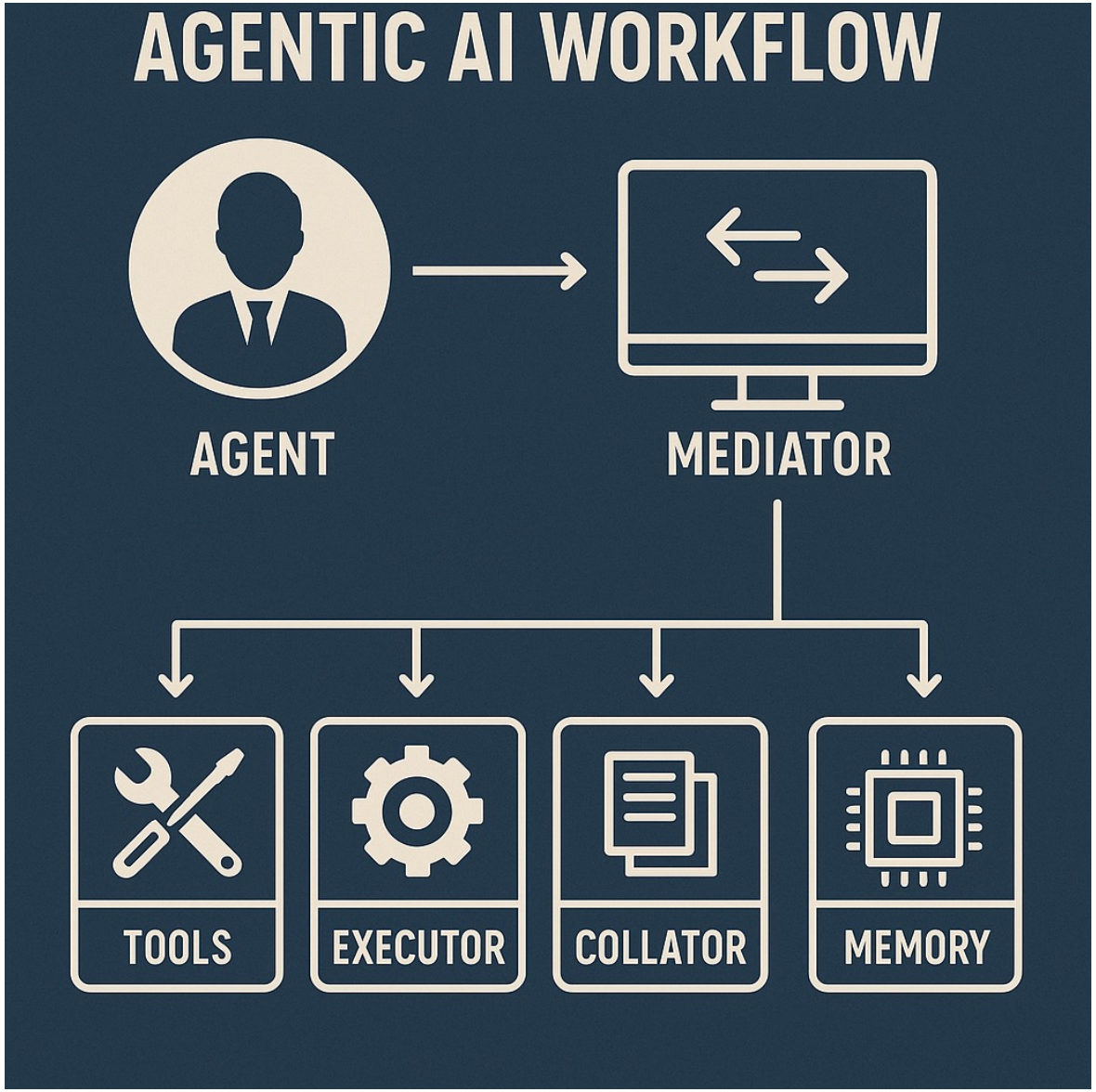
THE LATEST VERSION

OF API

TO STAGING

Summary

Component	What it does	Example
Agent	Takes the goal and initiates the process	“Deploy latest Flask API to staging”
Planner	Breaks the task: Run tests → Build Docker → Push image → Deploy	Uses planning logic to determine best flow
Executor	Executes shell commands, Git commands, Docker builds, AWS CLI	docker build, aws ecs update-service, etc.


AGENTIC AI WORKFLOW
AGENT
MEDIATOR
TOOLS
EXECUTOR
COLLATOR
MEMORY

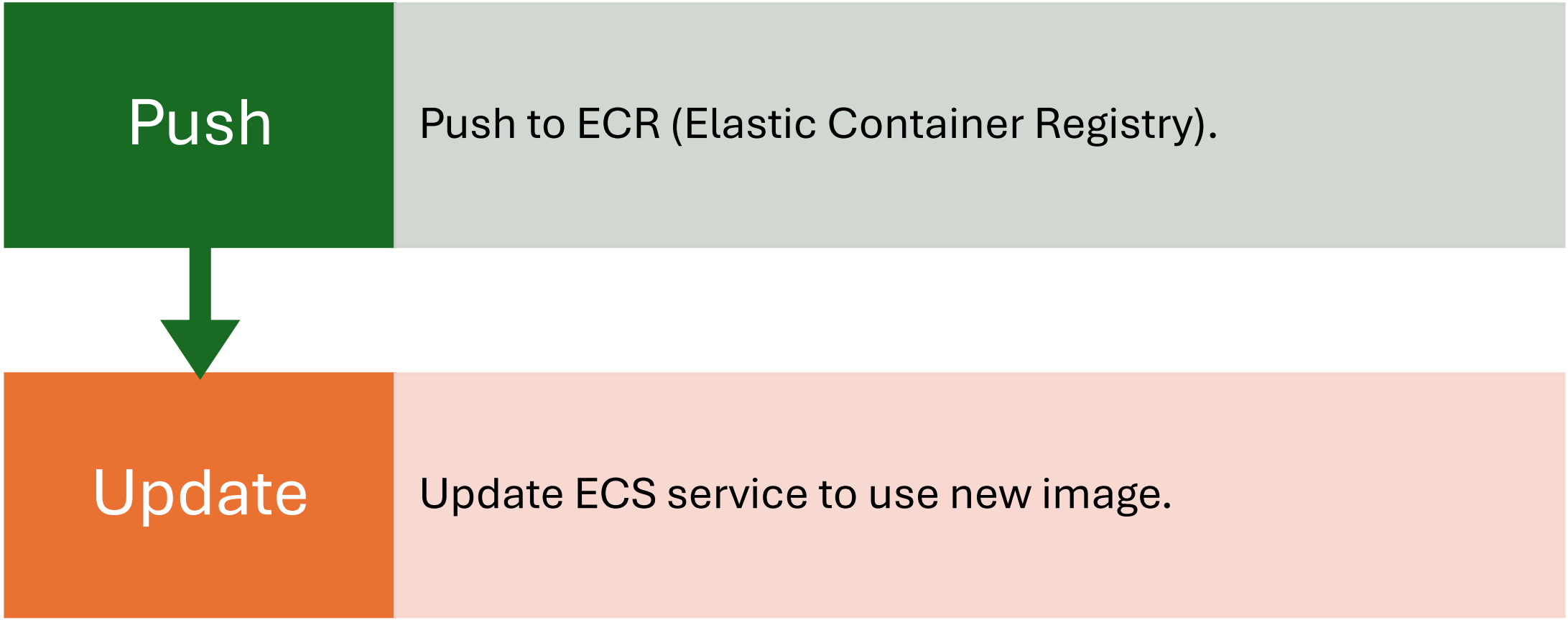
Planner

Step 1: Pull latest code from GitHub.

Step 2: Run pytest for test validation.

Step 3: Build Docker image.

Planner


Push
Push to ECR (Elastic Container Registry).
Update
Update ECS service to use new image.

Executor

Runs each command:

git pull origin main

pytest

Executor

docker build -t flask-api .

docker push <ECR repo>

aws ecs update-service

...

Agent

Monitors output and errors.

If a test fails,

updates planner

to rerun after fix.

Agent

SENDS

SUCCESS/FAILURE REPORT

TO SLACK OR EMAIL.

Infrastructure Team Role

Provide APIs, credentials, and environments.

Agent uses this info

To deploy

without human involvement.

Example

IF INFRA TEAM HAS SET UP TERRAFORM,

THE AGENT CAN USE IT

TO PROVISION RESOURCES DYNAMICALLY.

Key Components of Agentic AI

Component	Role
Agent	The brain that manages the goal and coordinates tasks
Planner	Breaks down the goal into steps (strategy builder)
Executor	Executes each step (action taker)

Benefits

Role	Benefit
Project Manager	Get faster and more reliable releases, reduced coordination overhead.
Delivery Manager	Accelerate timelines, reduce dependency on manual CI/CD steps.
Infra Team	Standardized, secure, and automated infra usage with reduced load.

Use Case – Flask API Deployment

Surendra Panpaliya

Goal

Deploy

the latest version of

Flask API to staging

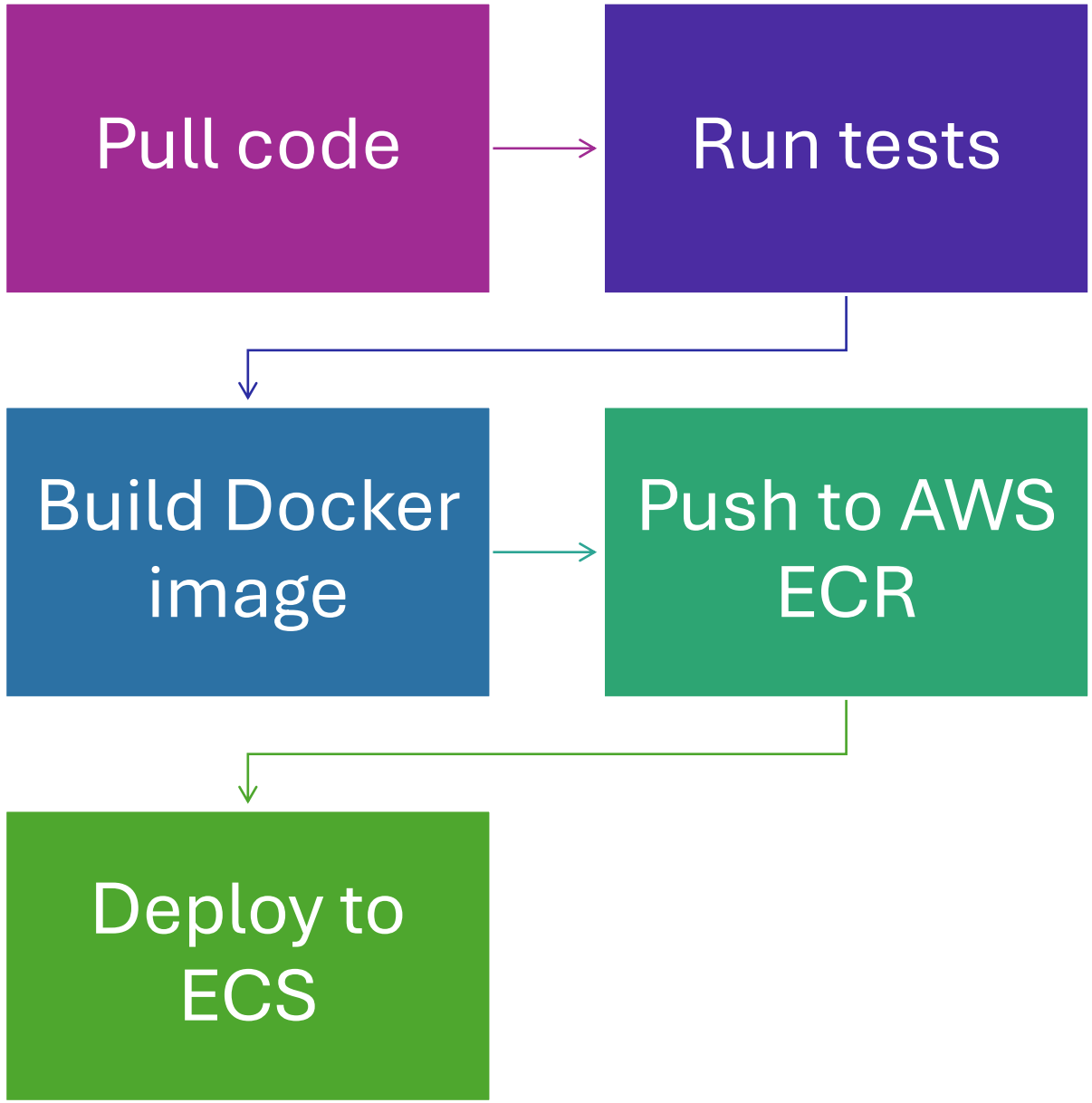
Agent

UNDERSTANDS

THE TASK

DELEGATES

Planner


Pull code
Run tests
Build Docker image
Push to AWS ECR
Deploy to ECS

Executor

Runs actual commands

docker build,

aws ecs update-service

Outcome

Fully automated

CI/CD loop

with minimal

human input

How It Works?

Agent receives goal

Planner creates task flow

Executor performs actions

Agent handles errors, retries, and reporting

Benefits by Role

Role	Benefits
Project Manager	Faster, consistent deliveries; real-time updates
Delivery Manager	Reduced risk and cycle time; improved predictability
Infra Team	Less manual overhead; agent uses APIs, scripts securely

Tools & Frameworks

- LangChain + LangGraph
- AutoGen by Microsoft
- Azure OpenAI / OpenAI GPT
- Milvus / Redis

Open-Source Frameworks at a Glance

Tool	Built By	Ideal For
AutoGen	Microsoft	Multi-agent chat workflows
LangGraph	LangChain Team	Graph-based decision pipelines
CrewAI	Community	Role-based agent teams

📚 Reference Links

Topic	Link
OpenAI AutoGen Agents	https://github.com/microsoft/autogen
LangChain Agents & Executors	https://python.langchain.com/docs/introduction/
LangGraph Agentic Framework	https://langchain-ai.github.io/langgraph/

What is LangChain?

A powerful **framework**

for building applications

using Large Language Models (LLMs).

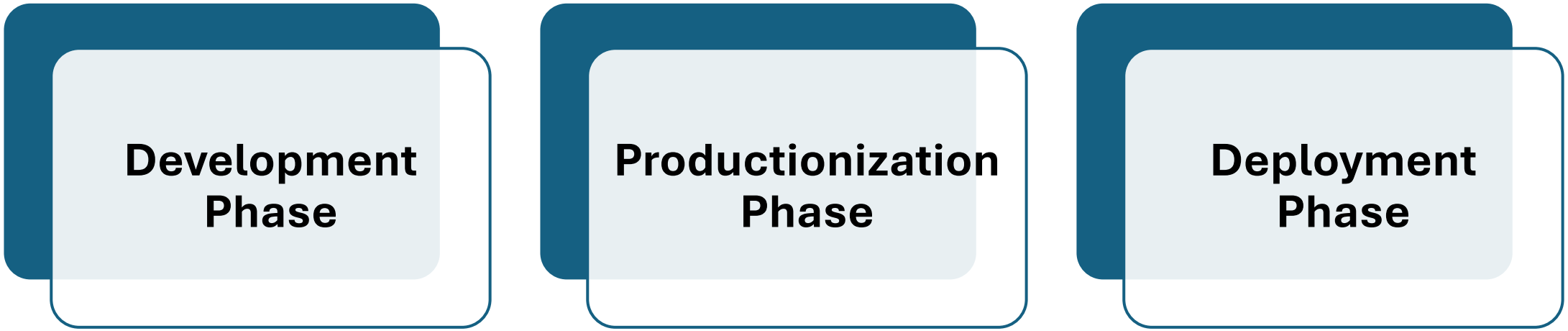
What is LangChain?

SIMPLIFIES
EVERYTHING

FROM DEVELOPMENT
TO DEPLOYMENT

OF LLM-BASED
SOFTWARE SYSTEMS.

Key Capabilities at a Glance



1. Development Phase

Use **open-source components**

Third-party tools

to build **LLM applications**.

1. Development Phase

- **LangGraph**
- enables
- to create
- **stateful agents**

1. Development Phase

- Support
- Streaming outputs
- Human-in-the-loop feedback
- Tool usage coordination

2. Productionization Phase

LangSmith

Inspect and debug LLM applications

Monitor performance

Evaluate accuracy & quality

Continuously optimize models and prompts

3. Deployment Phase

Turn LangGraph apps

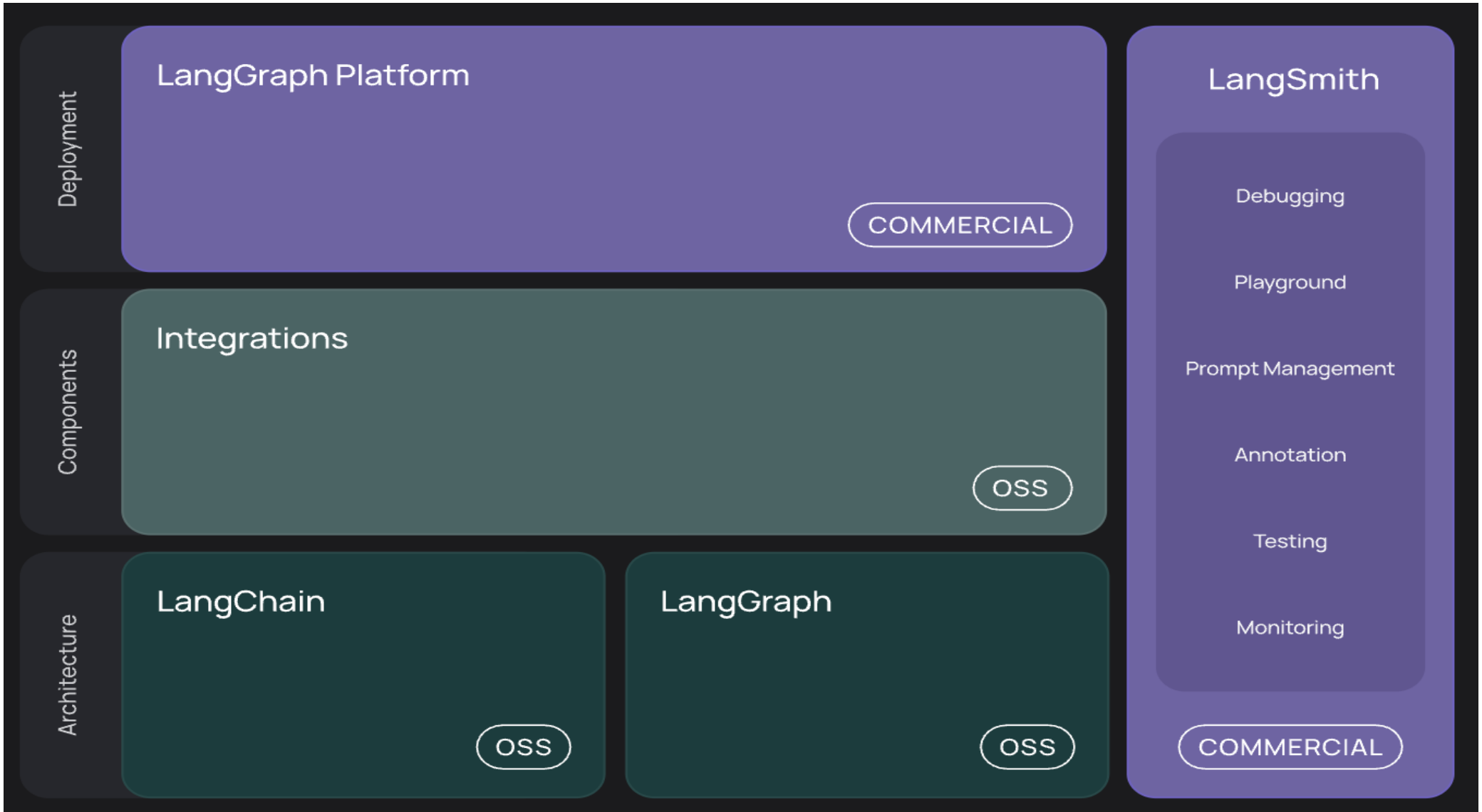
Into APIs

Production-ready AI Assistants

3. Deployment Phase

- Use **LangGraph Platform**
- for cloud-based hosting
- and orchestration

LangChain + LangGraph Architecture



Architecture Overview

Module	Purpose
langchain-core	Base interfaces & abstractions for LLMs, tools, and prompts
langchain	Contains higher-level components like agents, chains, and retrieval strategies

Architecture Overview

Module	Purpose
langchain-openai, langchain-anthropic,	Integration packages maintained by LangChain + provider teams
langchain-community	Third-party and community-contributed integrations

Architecture Overview

Module	Purpose
langgraph	Agent orchestration framework with support for memory, streaming, and persistence
LangSmith	Observability and evaluation platform for LangChain apps
LangGraph Platform	Cloud deployment & lifecycle management of LangChain applications

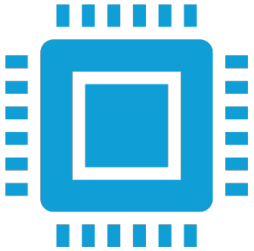
Reference

- https://python.langchain.com/docs/introduction/

Ecosystem Integration

LangChain integrates

with **hundreds of tools**,



models, and providers

GenAI Tools Providers

OpenAI,

Anthropic,

Hugging Face,

Cohere,

Google Vertex

Vector stores

Pinecone,

Chroma,

FAISS

Tools

Search APIs,

SQL databases,

file readers,

Python REPL

Cognitive Architecture Features

LangChain’s design

encourages building apps

that simulate thinking agents with:

Memory, Tool usage

Multi-step planning,

Retrieval-Augmented Generation (RAG)

Cognitive Architecture Features

Memory: Store past interactions

Tool usage: Perform calculations, search, look up info

Multi-step planning: Use planners, routers, chains

Retrieval-Augmented Generation (RAG)

Fetch relevant knowledge before answering

Summary of Key Points

Category	Summary
Framework Type	Agentic LLM application development
Core Components	Chains, Agents, Tools, LangGraph
Lifecycle Coverage	Build → Monitor (LangSmith) → Deploy (LangGraph Platform)

Summary of Key Points

Category	Summary
Extensibility	Integrates with major LLM providers and tools
Production Readiness	Enables streaming, persistence, tracing, and evaluation

Microsoft's AutoGen

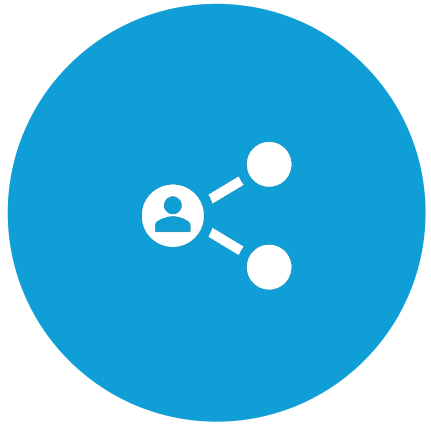
A guide to code-executing agents

TUTORIALS

What is AutoGen?

OPEN-SOURCE FRAMEWORK

FOR BUILDING LLM-POWERED



MULTI-AGENT SYSTEMS.

What is AutoGen?

Created by Microsoft,

in collaboration with Penn State

University of Washington.

What is AutoGen?

SIMPLIFIES

CREATING **AUTONOMOUS OR SEMI-AUTONOMOUS AGENTS**

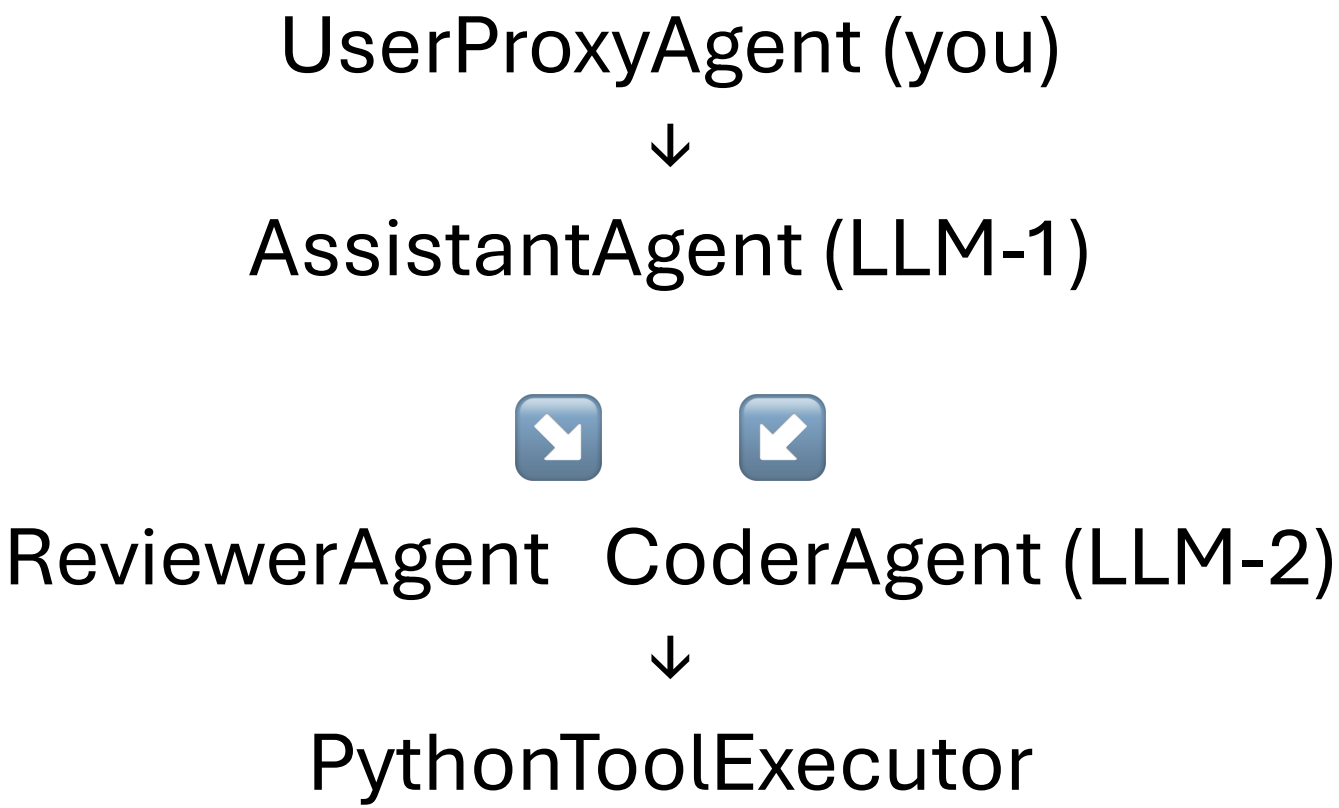
ENABLING **MULTI-AGENT CONVERSATIONS**

BUILDING **TOOL-USING, REASONING-CAPABLE WORKFLOWS**

Key Features of AutoGen

Feature	Description
Conversable Agents	Agents can talk to each other or humans
Tool Integration	Agents can call functions or run code (e.g., Python REPL, Search APIs)
Flexible Autonomy	Mix autonomous + human-in-the-loop agents
Customizable Workflows	Define conversation patterns and task topology
Modular Architecture	Compose agents using different LLMs and APIs

Multi-Agent Conversation Architecture



Multi-Agent Conversation Architecture

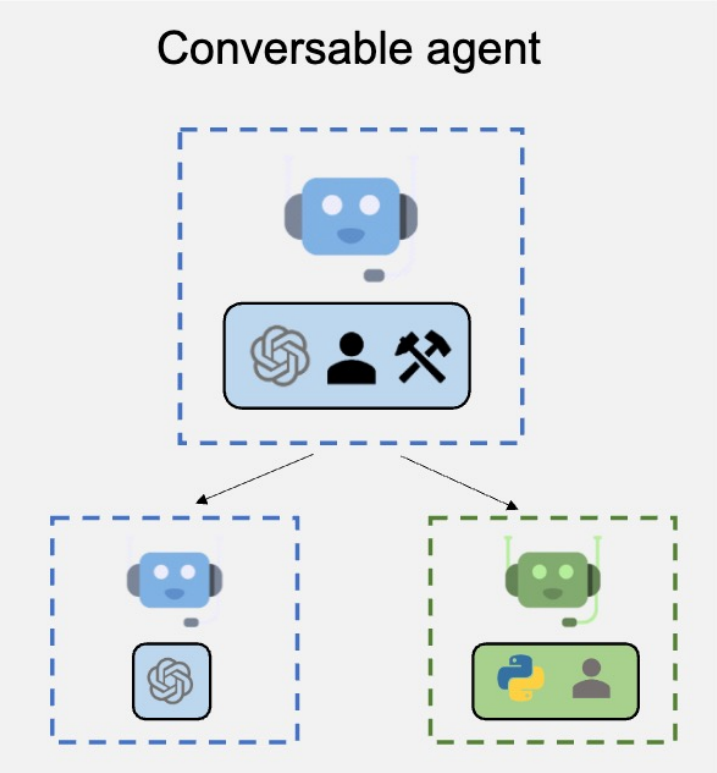
AutoGen’s conversation model supports:

1-to-1 (User ↔ Agent)

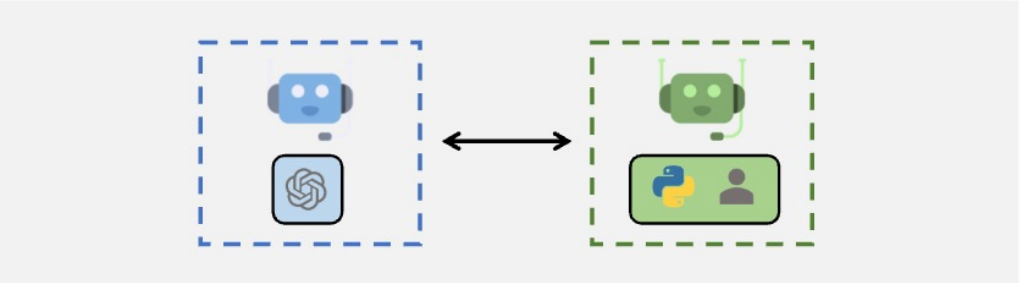
1-to-N (User ↔ Multiple Assistants)

N-to-N (Team of agents collaborating)

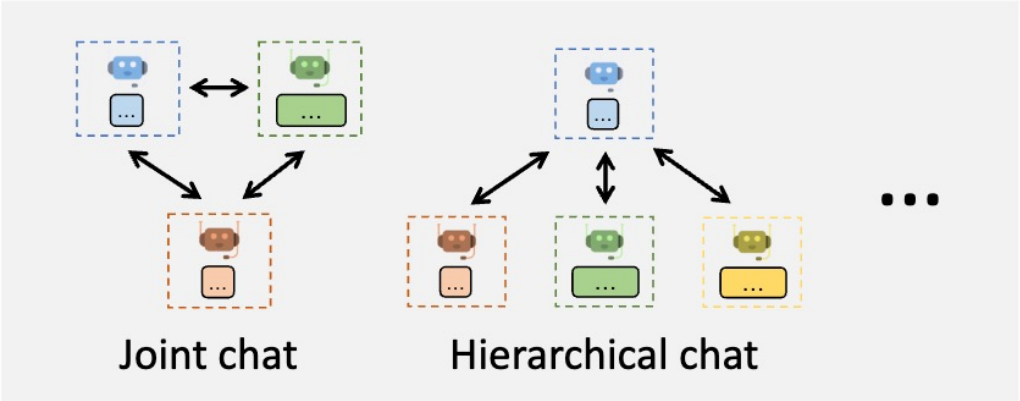
Multi-Agent Conversation Architecture



Agent Customization



Multi-Agent Conversations



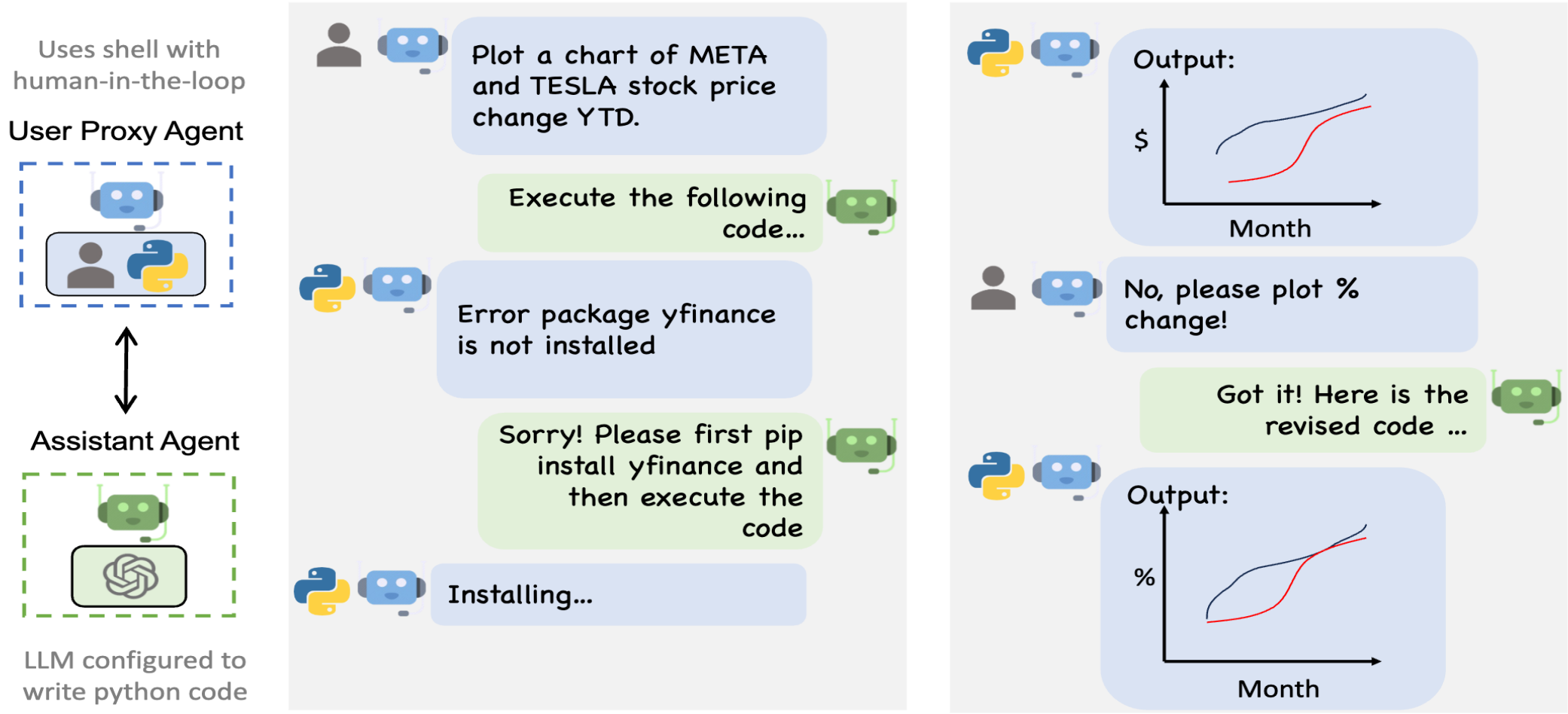
Flexible Conversation Patterns

Multi-Agent Conversation Architecture

Agents can:

- Call tools (e.g., Python code, APIs)
- Collaborate and debate
- Request human input

Multi-Agent Conversation Architecture



Real-World Example (Code Snippet)

```
from autogen import AssistantAgent, UserProxyAgent

llm_config = {"config_list": [{"model": "gpt-4", "api_key":
os.environ['OPENAI_API_KEY']}]}

assistant = AssistantAgent("assistant", llm_config=llm_config)

user_proxy = UserProxyAgent("user_proxy", code_execution_config=False)

user_proxy.initiate_chat(assistant, message="Tell me a joke about NVDA and
TESLA.")
```

Real-World Example (Code Snippet)

This simple example shows

how an LLM-powered assistant

interacts with a user,

but you can plug in

more agents and tools.

Benefits for Leadership

Role	Strategic Advantage
Project Manager	Automate documentation, task planning
Delivery Manager	Integrate agents for release validation, testing
CI/CD Engineer	Chain agents to build/test/deploy apps
AI Architect	Create flexible reasoning systems using multiple LLMs

Execution Options

No Code Execution → Only LLM responses

Local Execution → Run Python tools locally

Docker Execution → Run agent tools in isolated containers

Execution Options

Supports multiple use cases:

- Report generation
- Agent-led code review
- Data analysis agents
- CI/CD automation

Summary

AutoGen empowers businesses

to build next-gen intelligent applications using:

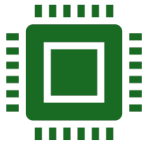
Multiple collaborative AI agents

Secure, auditable, and tool-using logic

Autonomous or human-guided workflows

Summary

Perfect for:



DevOps,

MLOps,

AI Assistants,

Automation

Resources & Links

Video tutorial on AutoGen Studio v0.4 (02/25)

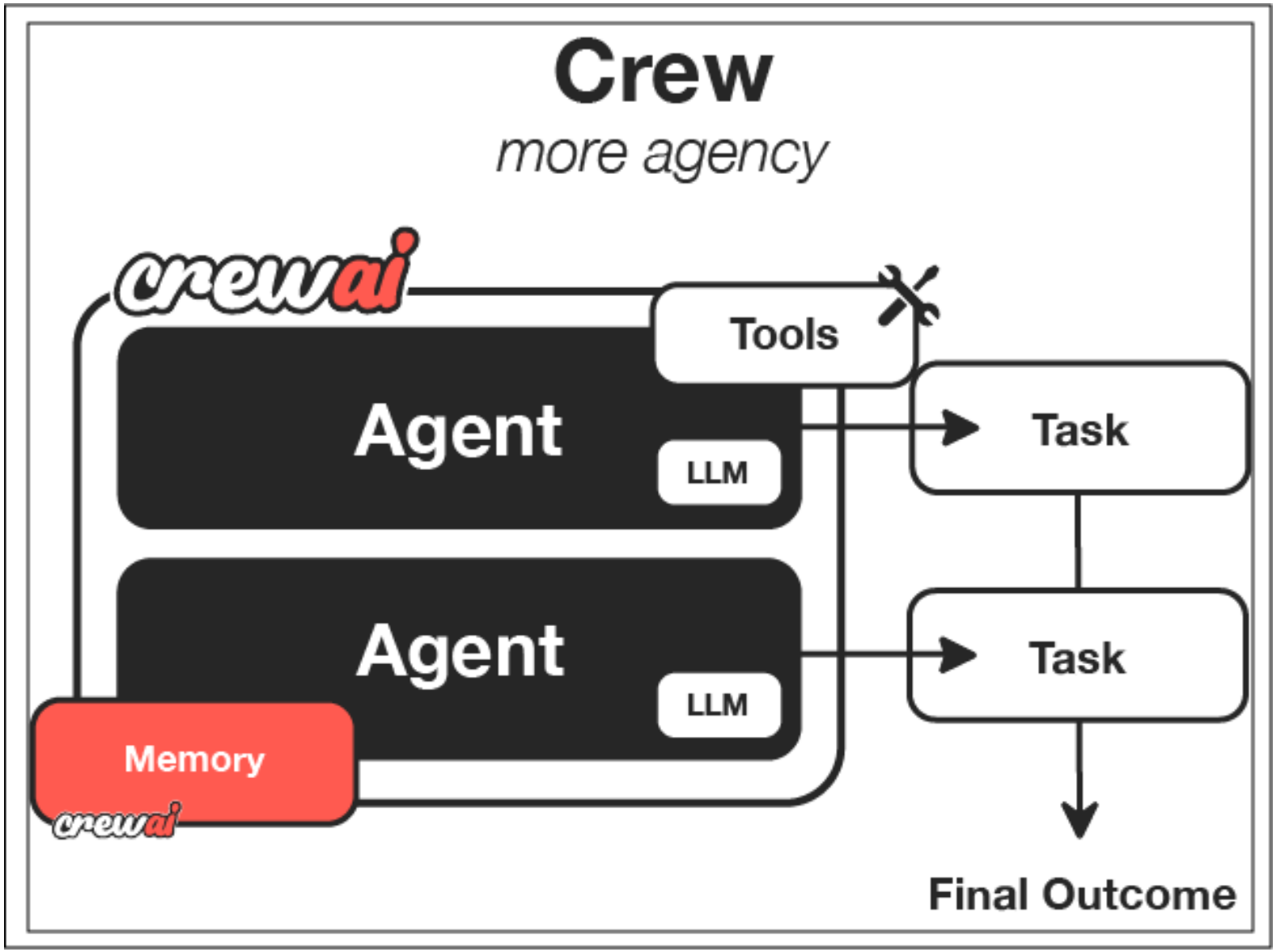
https://youtu.be/oum6EI7wohM

Resources & Links

GitHub: https://github.com/microsoft/autogen

Docs: https://microsoft.github.io/autogen/

Multi-Agent Guide: https://microsoft.github.io/autogen/docs/Concepts/Multi-Agent-Chat/


Crew
more agency
crewai
Tools
Agent
LLM
Agent
LLM
Memory
crewai
Task
Task
Final Outcome

👬 CrewAI

CrewAI is a new open-source framework

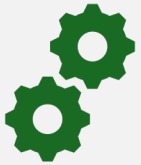
for building **crews (teams of AI agents)**

that work together with

specific roles, goals, and tools.

How It Works?

Define agents (like teammates)



Assign tools (e.g., search, code exec, APIs)

Define mission → Crew collaborates to solve

Example in DevOps

Goal: Set up cloud infra for a new microservice

🤖 CrewAI simplifies

collaboration between

agents as if they were

human DevOps teammates.

CrewAI Example in DevOps

Crew Member (Agent)	Role	Tool Used
InfraArchitect	Chooses right AWS services	LangChain tool, docs
Terraformer	Writes Terraform code	Python shell
Verifier	Validates resource config	Cloud API, logging

Reference

CrewAI Flows | Sales Pipeline Flow Demo

https://www.youtube.com/watch?v=MTb5my6VOT8

Repo: https://github.com/joaomdmoura/crewAI

Summary Comparison Table

Tool	Ideal For	Approach	Real Use Case
AutoGen	Chat-like agent conversations	Conversational	Multi-agent CI/CD assistant
LangGraph	Workflow/state-based agents	Graph + memory	Conditional deployment flow
CrewAI	Role-based collaboration	Structured teams	Infra setup, DevOps collaboration

Benefits for IT Leaders

Role	Value Addition
Project Manager	Define outcomes, let agents execute and monitor
Delivery Manager	Increase speed, reduce human errors
Infra Team	Automate provisioning, testing, rollback, scaling

Final Thought

Agentic AI tools like

AutoGen, LangGraph, and CrewAI

enable **AI-first automation**,

beyond traditional scripting or pipelines.

Final Thought

AUTOGEN, LANGGRAPH, AND CREWAI

ARE YOUR **DIGITAL TEAM MEMBERS**

COLLABORATIVE, EFFICIENT, AND INTELLIGENT

What Are Ethical & Governance Considerations in AI?

Surendra Panpaliya

Ethics

ENSURING THE **RIGHT THING** IS DONE:

FAIR, SAFE, PRIVATE,

UNBIASED, AND EXPLAINABLE.

Governance

ENSURING THE **RIGHT PROCESS** IS FOLLOWED

TRANSPARENT, AUDITABLE,

SECURE, AND COMPLIANT.

Why It Matters: Real-World Risks

Scenario	Risk	Why Governance & Ethics Matter
AutoGen agent pushes code	Bug or bias introduced	Was the change reviewed or audited?
LangGraph agent deploys ML model	Model leaks user data	Was privacy protected?
CrewAI provisions AWS services	Misconfigures IAM policy	Who approved the change? Is it traceable?

Key Ethical Principles in AI

Principle	Description	Example
Transparency	Know what the AI did and why	Agent logs each action, decision, and error
Fairness	Avoid bias or unfair outcomes	ML model avoids gender/race bias in feature
Privacy	Protect sensitive data	No raw data stored or leaked during deployment

Key Ethical Principles in AI

Principle	Description	Example
Accountability	Who is responsible?	All agent actions linked to owner or role
Security	Prevent malicious use or leaks	Access tokens encrypted, secrets in Vault

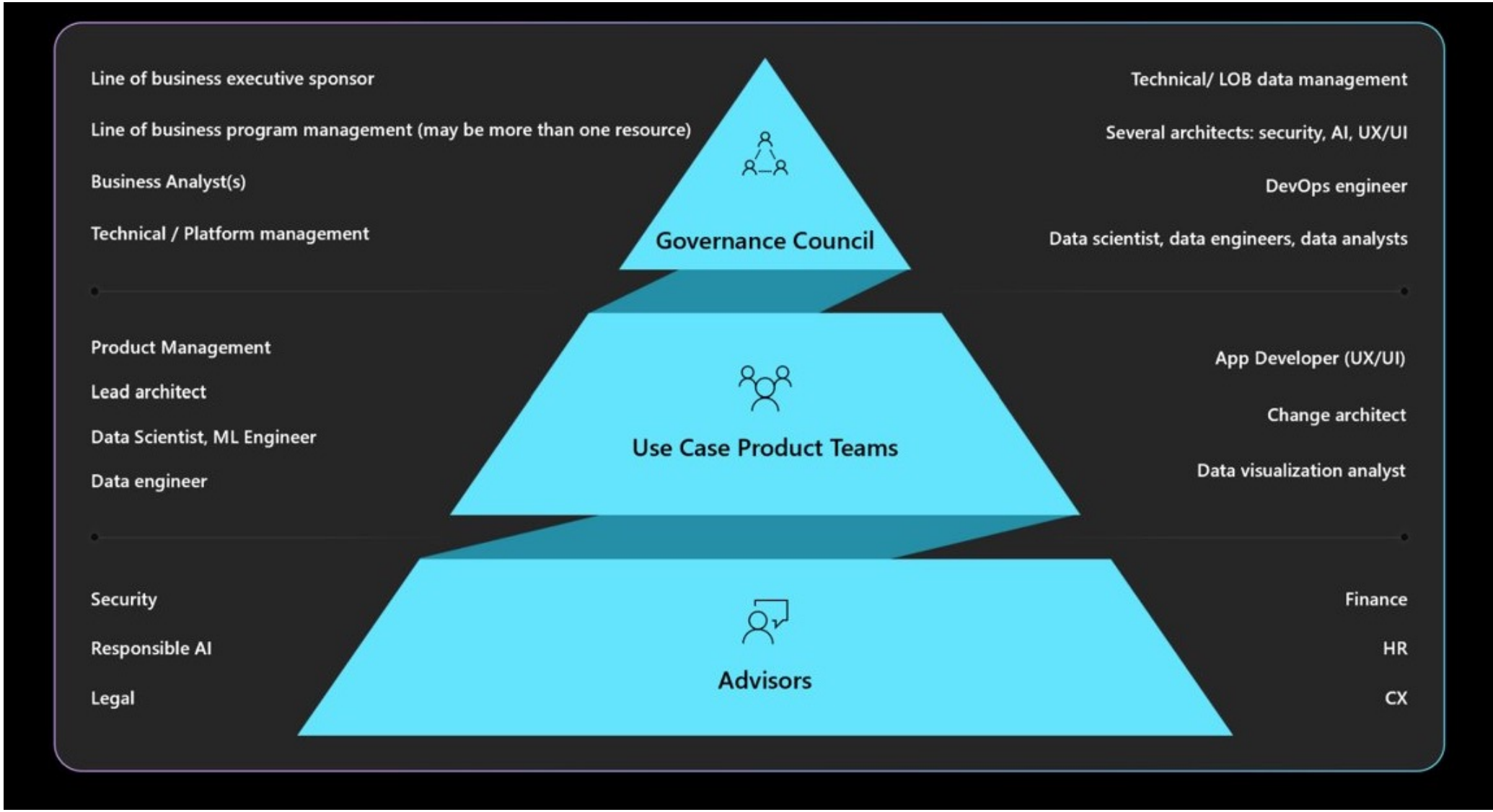
📌 Reference:

- Microsoft Responsible AI Principles
- EU AI Act: Risk-based Governance
- OECD AI Principles

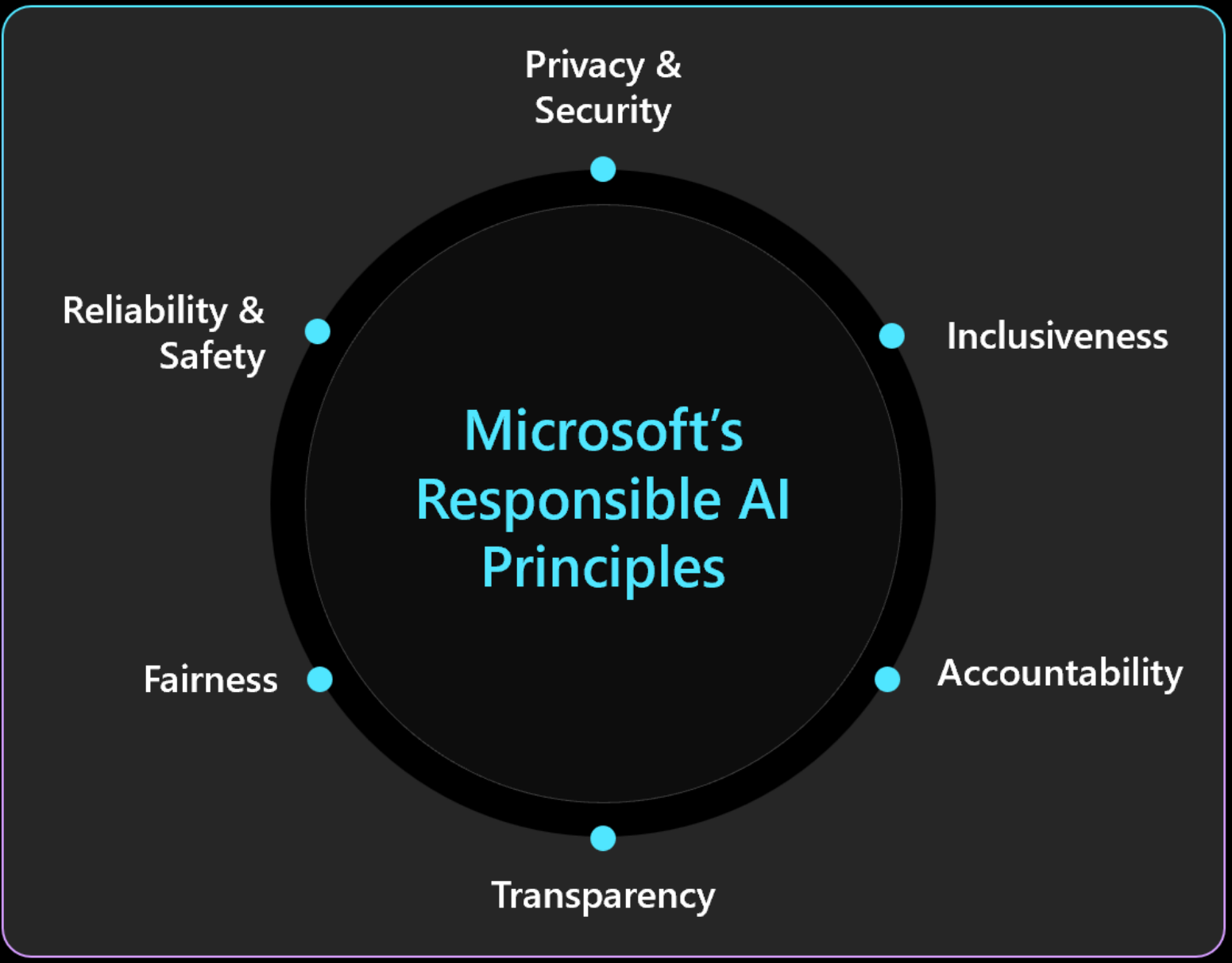
Governance Framework: Core Components

Governance Area	Example Tools	Explanation
Audit Trails	LangSmith, MLflow	Tracks each decision/output by the agent
Version Control	GitHub, DVC	Keeps history of code, models, prompts
Role-based Access	AWS IAM, Vault	Limits who/what can run agent workflows
Monitoring	Azure Monitor, Prometheus	Detect anomalies, errors in AI executions
Compliance Logging	S3, CloudTrail, SOC2 logs	Needed for external/regulatory audits

Example Diagram: AI Governance Lifecycle



Microsoft's Responsible AI principles



Building blocks to enact principles

- Tools and processes
- Training and practices
- Rules
- Governance

Example Diagram: AI Governance Lifecycle

https://learn.microsoft.com/en-us/azure/well-architected/ai/responsible-ai

Ethical Governance in Software Deployment

Let’s walk through a

real-time software deployment process

using Agentic AI

(AutoGen or LangGraph)

Ethical Governance in Software Deployment

🎯 **Goal:** **Deploy new version of** **a Flask API using AutoGen**

Step-by-step AI Agent Workflow

Agent (Planner) breaks down goal:

Pull latest code

Run unit tests

Build Docker image

Deploy to staging

Governance & Ethics Checks

Step	Risk	Governance Control	Ethical Check
Pull code	Wrong repo, malicious code	Use GitHub repo hash + approval trigger	Is this change auditable?
Run tests	Failing tests skipped	CI rule: Block deploy if test < 80% pass	Did AI skip edge-case tests?

Governance & Ethics Checks

Step	Risk	Governance Control	Ethical Check
Build image	Secrets exposed in Dockerfile	Vault manages env vars and secrets	Is PII or API key leaking in logs?
Deploy to staging	Wrong config in ECS	Agent must validate against policy schema	Is the config biased toward specific users/locations?

Tools Used for Governance:

- **LangSmith**: Tracks agent steps, reasoning, errors
- **MLflow/DVC**: Model + prompt version control
- **Vault by HashiCorp**: Secrets + token management
- **OpenAI Moderation API**: Filters toxic/unethical output

What is Vault?

Vault is a centralized,

secure **secrets management and**

privileged access platform.

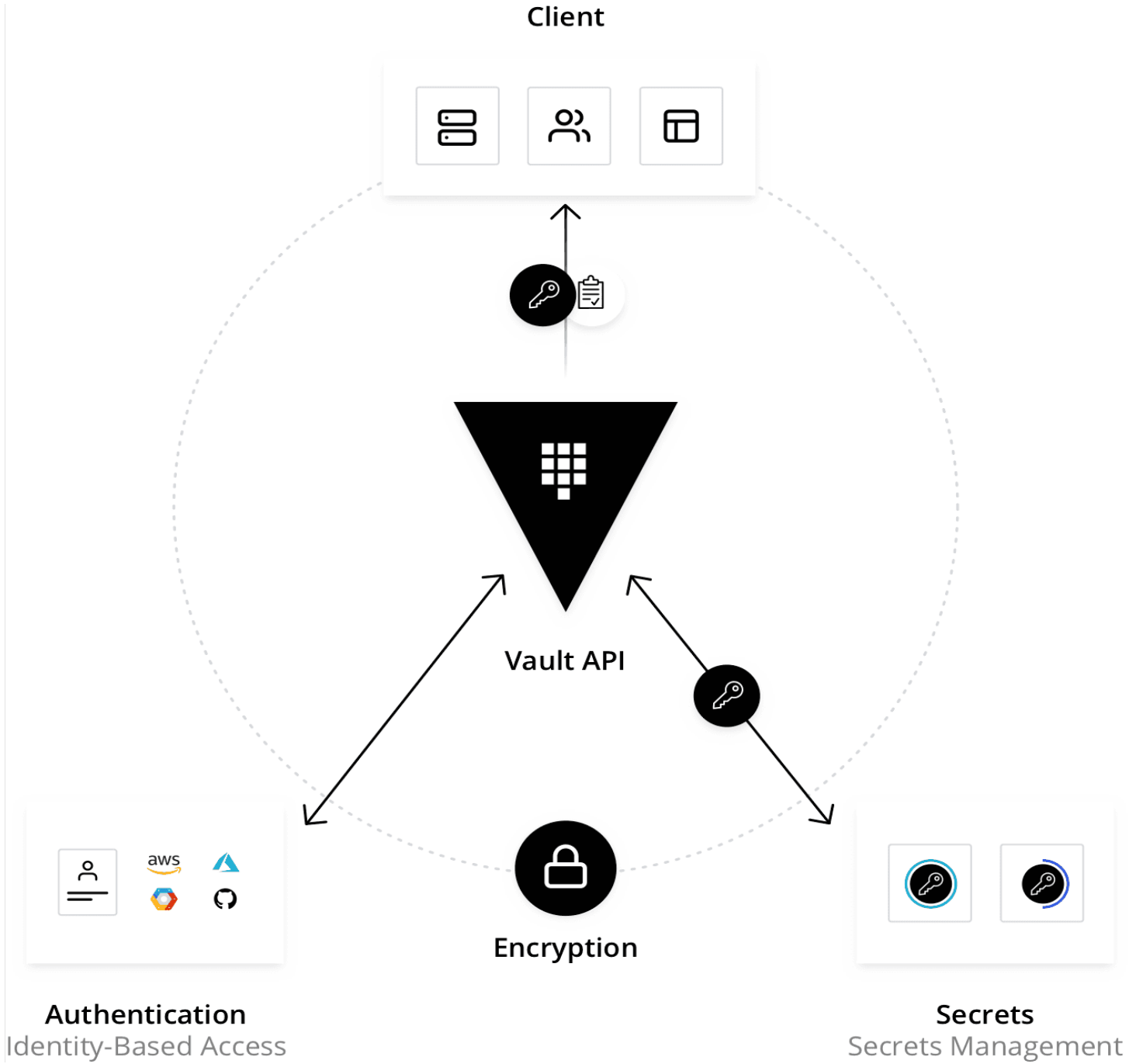
What is Vault?

Supports **cloud, on-prem,**

and hybrid deployments.

Offers strong **auditing, encryption,**

and integration capabilities.


Client
Vault API
Encryption
aws
Authentication
Identity-Based Access
Secrets
Secrets Management

What is Vault?

Secure, store, and tightly control access

to tokens, passwords, certificates,

encryption keys for protecting secrets, and

other sensitive data using a UI, CLI, or HTTP API.

Why Use Vault?

Modern apps depend on secrets:

API keys, credentials,

tokens,

encryption certs

Why Use Vault?

Vault ensures:

Centralized secret storage

Granular access control

Audit logging for compliance

Dynamic secrets for databases and cloud apps

Core Use Cases

Use Case	Description
Static Secret Management	Store and retrieve credentials securely
Certificate Management	Automate issuing & revoking TLS certs
Identity & Auth Integration	Integrate with LDAP, Azure AD, GitHub etc.

Core Use Cases

Use Case	Description
Dynamic Secrets	Generate DB or cloud credentials on demand
Compliance & Auditing	Track access and enforce policy rules

Plugin-Based Architecture

- Vault is modular, powered by a **plugin ecosystem**:
- **Auth Plugins**: LDAP, GitHub, Azure, AWS
- **Secret Plugins**: Store or generate static and dynamic secrets
- **Database Plugins**: Manage ephemeral credentials for PostgreSQL, MySQL, etc.
- Supports **custom plugin development** for unique workflows

Access & Authorization Flow

- Client authenticates (token, LDAP, cloud provider)
- Vault maps client to an **internal identity**
- Access policies control what paths/secrets are allowed
- Vault logs every access attempt (success or failure)

Storage Backend Options

Storage Type	HA Support	Description
Integrated	✅	Built-in, encrypted, highly available
File System	❌	Local-only (not for production)
External	⚠️ Maybe	Supports AWS, Azure, GCP, MySQL
In-Memory	❌	Volatile; for testing/dev environments

HCP Vault Dedicated

Managed Vault on

HashiCorp Cloud Platform (HCP)

Same binary as

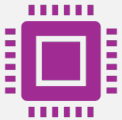
self-managed Vault Enterprise

HCP Vault Dedicated

Ideal for:

Quick onboarding

No infra management



Auto-scaling & high availability

HCP Vault Dedicated

- Try it here:
- https://cloud.hashicorp.com/products/vault

When Not to Use Vault?

Small teams with **basic secret storage needs**

Teams without infra management bandwidth

Consider **HCP Vault Dedicated** instead for simplicity

How to Get Started?

- Download binary: https://developer.hashicorp.com/vault/downloads
- Install via package manager (Linux, Mac, Windows)
- Use **Vault CLI, API, or UI**

How to Get Started?

- Explore GitHub repo for source:
- https://github.com/hashicorp/vault

References

Vault UI for secrets management:

Vault by HashiCorp

https://developer.hashicorp.com/vault

Summary

Vault is the industry-standard solution for:

Securing **secrets, access, and identities**

Enforcing **compliance**

Supporting **zero trust architectures**

Summary

Perfect for organizations prioritizing:

DevSecOps

Cloud-native workloads

Regulatory requirements (SOC2, HIPAA, PCI-DSS)

Modern secrets management starts with Vault.

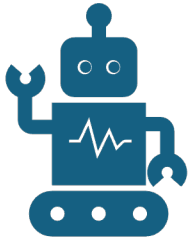
Stakeholder-Specific Guidance

Surendra Panpaliya

🎯 For Senior Project Managers

Ensure **goal alignment** and

ethical responsibility



for AI-based automations.

🎯 For Senior Project Managers

SET CLEAR

SUCCESS/FAILURE CRITERIA

WITH ROLLBACK STEPS.

🎯 For Senior Project Managers

ENSURE AGENT ACTIVITIES ARE

TRACEABLE AND REVIEWABLE.

🎯 For Delivery Managers

ENFORCE

APPROVAL GATES

BEFORE

CRITICAL DEPLOYMENT.

🎯 For Delivery Managers

Monitor

agent performance

with dashboards

(Prometheus/Grafana)

🎯 For Delivery Managers

Embed

ethical guardrails

block known toxic behaviors.

🎯 **For IT Infra Team**

Implement

RBAC policies

for agent

execution environments.

🎯 For IT Infra Team

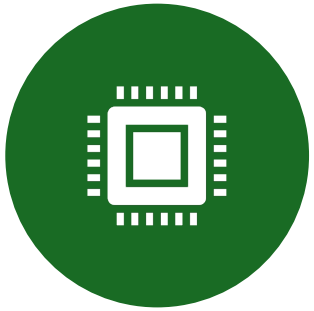
Use

container scanning tools

like Trivy or Snyk.

🎯 For IT Infra Team

ENSURE



INFRASTRUCTURE-AS-CODE (IAC)

IS AUDITED

(TERRAFORM, PULUMI).

🛠️ Key Tools You Should Explore

Tool/Platform	Purpose
LangSmith	Agent tracing, debugging, audit
MLflow	Model governance, experiment tracking
Vault	Secrets & access control
OpenAI Risk Tools	Prevent harmful outputs
DVC (Data Version Control)	Versioning for ML assets

🎓 Best Practices Summary

Log	Always log and version agent actions
Use	Use moderation filters
Output	for unsafe or biased output
Avoid	Avoid hard-coded secrets
Use	use Vault or AWS Secrets Manager

🎓 Best Practices Summary

REVIEW **AGENT DECISIONS**

LIKE YOU WOULD CODE PRS

PREPARE **COMPLIANCE REPORTS**

FROM AUDIT LOGS

Further Learning Resources

- AI Governance Framework – WEF
- Responsible AI Practices – Google
- Azure Responsible AI Governance

Final Thought

AI doesn’t just need

to be intelligent

it needs to

be responsible.

Happy Learning!! Thanks for Your Patience ☺

Surendra Panpaliya

GKTCS Innovations